

MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering

Croitoru Artemie
Group: FAF252

Report

for the Laboratory Work no. 1

for the course "Object-Oriented Programming"

Chişinău – 2026

Logging Library and Parentheses Validator

1 Task

Build a small logging library and use it to solve a programming problem. The laboratory work is implemented in Java.

The solution must:

- contain a `Logger` class with a name, a configurable minimum level and a configurable format string;
- contain a `ParenthesesValidator` class that checks whether a string of parentheses is valid;
- use the logger inside the validator to report the events that happen during the validation;
- be tested with all the inputs given in the handout.

2 Problem Statement

2.1 Task 1 — the Logger class

Each logger has a name that belongs to the logger object. The class also has two static members, `level` and `format`, shared by all logger objects, and both of them can be changed at runtime.

Four log levels are implemented in the enum `LogLevel`, ordered by severity: TRACE $<$ INFO $<$ WARN $<$ ERROR. The configured level is the minimum level that is printed. For example, with the level set to INFO the messages INFO, WARN and ERROR are printed, and TRACE messages are hidden.

The format string supports the following placeholders; every other character is printed unchanged:

Placeholder	Meaning
`%t`	current time
`%m`	the log message
`%n`	the name of the logger
`%l`	the level of the message
`%r`	random message of the current level (extra credit)

The default format used in my program is `[%t] [%l] %n:  %m`.

2.2 Task 2 — the parentheses problem

A string is valid if every opening parenthesis `(` has a matching closing parenthesis `)` and the parentheses are correctly ordered. The method `isValid` receives a string and returns `true` or `false`. The required tests are:

Input	Expected result
`()`	true
`)(((())))`	false
`(`	false
`(())((()())())`	true

2.3 Task 3 — using the logger

The validator creates a logger named `Parentheses` and logs: TRACE when an opening parenthesis is found, INFO when a matching pair is found, INFO for the final result, WARN when a closing parenthesis has no opening one, and ERROR when the whole input is invalid.

3 Program Structure

The program has four files: the enum `LogLevel`, the classes `Logger` and `ParenthesesValidator`, and the class `Main` with the test runner.

- `Logger` is responsible for output. It decides whether a message must be shown (comparing its level with the configured one), builds the final text from the format string and prints it with `System.out.println`. The methods `trace`, `info`, `warn` and `error` are shortcuts that call `log` with the right level.
- `ParenthesesValidator` is responsible only for the validation. It walks through the string and reports what it finds, but it never prints anything by itself.
- `Main` runs the tests and changes the level and the format to show how the logger behaves.

The two classes collaborate in the following way: the validator creates a `Logger` named `"Parentheses"` and calls it every time something important happens. This is a dependency (the validator *uses* the logger), not inheritance. Figure 1 shows the structure.

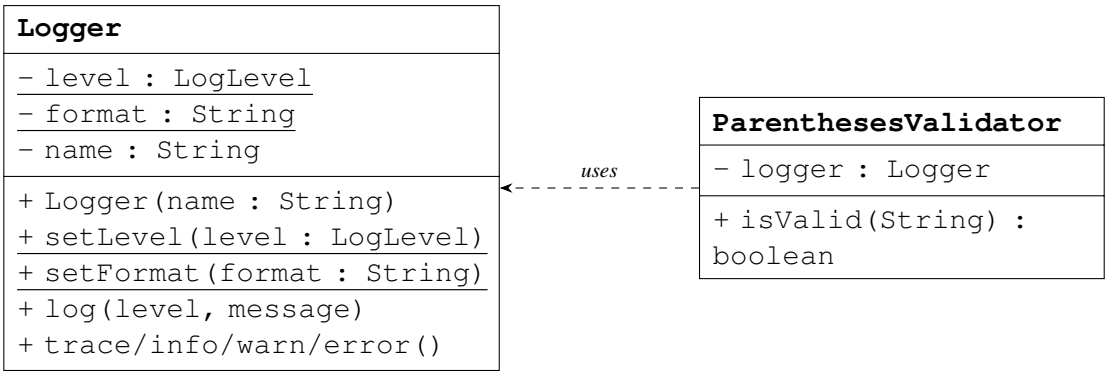


Figure 1: UML diagram of the classes (underlined members are static)

4 Classes and Objects

In my implementation a class is a blueprint that describes which data and which behavior an object has. I wrote three classes, `Logger`, `ParenthesesValidator` and `Main`, and one enum, `LogLevel`.

An example of an object created from a class is the logger used by the validator:

```
1 public class ParenthesesValidator {
2     private final Logger logger = new Logger("Parentheses");
3     // ...
4 }
```

The state stored by each object:

- a `Logger` object stores its `name`;
- a `ParenthesesValidator` object stores only a reference to its `logger`. The result of `isValid` is not stored, it is returned to the caller.

5 Static and Instance Members

5.1 Static members

`level` and `format` are static members of `Logger`. They are static because the minimum level and the output format are settings of the whole application, not of one particular logger. If I want to hide the trace messages, I want that for every logger. The constants `TIME_FORMAT` and `RANDOM_MESSAGES` and the object `random` are also static, because they are the same for all loggers.

5.2 Instance member

The member `name` belongs to each logger object. Two loggers can have different names (for example `Parentheses` and `Demo`) while still sharing the same level and format.

```java
public class Logger {
   // static: shared by all loggers
   private static LogLevel level = LogLevel.INFO;
   private static String format = "[%t] [%l] %n: %m";

   // instance: every logger has its own name
   private final String name;

   public Logger(String name) {
      this.name = name;
   }

   public static void setLevel(LogLevel newLevel) {
      level = newLevel;
   }
   // ...
}
```

5.3 Changing the level

When the level is changed with `Logger.setLevel(...)`, it affects every logger instance. The reason is that the value is stored only once, in the class `Logger`, and each instance reads it from there when `log` is called. There is no copy of the level inside the objects. For example, after `Logger.setLevel(LogLevel.ERROR)` both a `Parentheses` logger and a `Demo` logger print only errors.

6 Methods and Responsibilities

6.1 The log method

The method `log` does three things: it compares the severity of the message with the configured level and stops if the message is below it, it builds the text by replacing the placeholders, and it prints the result. The severity is the position of the constant in the enum, given by `ordinal()` (TRACE is 0, ERROR is 3).

```java
public void log(LogLevel msgLevel, String message) {
   // ignore messages that are less severe than the configured level
   if (msgLevel.ordinal() < level.ordinal()) {
      return;
   }
   System.out.println(buildText(msgLevel, message));
}
```

The method `buildText` goes through the format string only once, character by character. When it finds a `%` followed by a known letter, it appends the value of the placeholder, otherwise it appends the character unchanged. Because the format string is read only once, a text like `%n` written inside the message is not replaced a second time.

```java
private String buildText(LogLevel msgLevel, String message) {
```

```
    String result = "";
    int i = 0;
    while (i < format.length()) {
        char c = format.charAt(i);
        String value = null;
        if (c == '%' && i + 1 < format.length()) {
            value = getValue(format.charAt(i + 1), msgLevel, message);
        }
        if (value != null) {
            result += value;
            i += 2; // skip the % and the letter
        } else {
            result += c; // normal character, print it unchanged
            i++;
        }
    }
    return result;
}
```

6.2 Why the validator uses a logger

The validator should care only about checking parentheses. If it also handled levels and formats, the two jobs would be mixed and the code would be hard to read and to change. With a separate `Logger` the format or the level can be changed in one place and the validator stays the same. The logger can also be reused in other programs.

6.3 A call between objects

Inside `isValid` the validator calls its logger, and the logger's shortcut method calls `log`:

```
logger.trace("Opening parenthesis found at index " + i); // validator ->
    logger

public void trace(String message) { // inside Logger
    log(LogLevel.TRACE, message);
}
```

7 Algorithm of the Validator

The validator uses a stack (`java.util.Stack`) that keeps the index of every opening parenthesis that is not matched yet. The string is read once, character by character:

- for `(` the index is pushed on the stack (and a TRACE message is logged);
- for `)`, if the stack is empty, there is nothing to match, so the string is invalid (a WARN message is logged); otherwise the top of the stack is removed and the pair is logged with INFO;
- at the end, if the stack is not empty, some `(` was never closed, so the string is invalid.

The time complexity is $O(n)$ and the extra memory is $O(n)$ in the worst case, where n is the length of the string. The block diagram is shown in Figure 2.

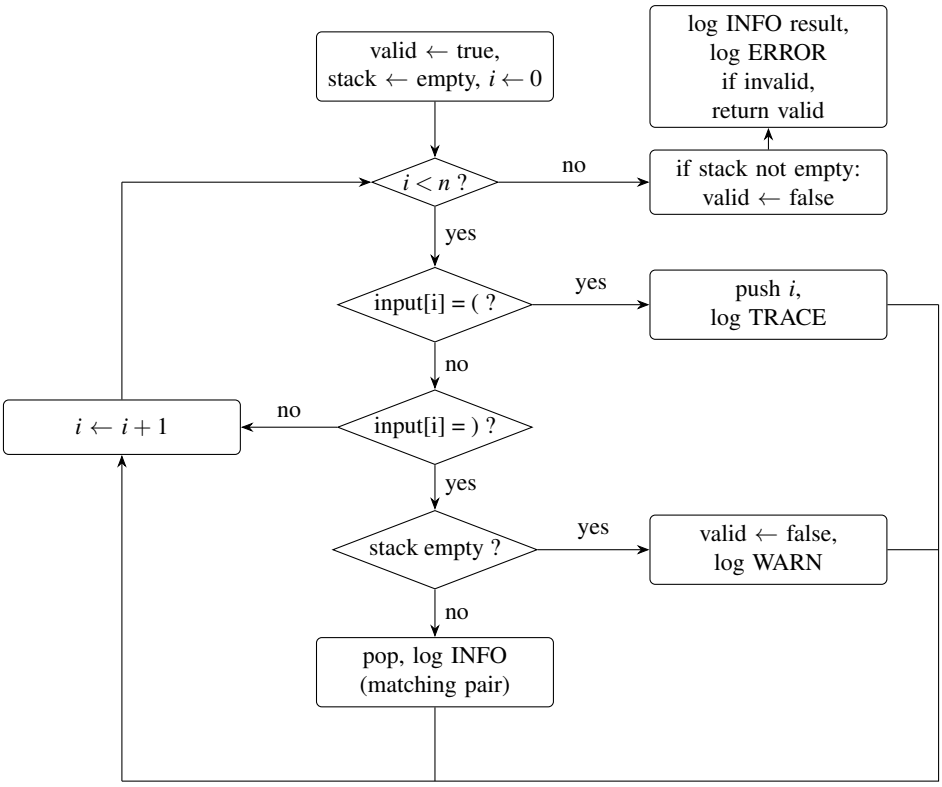


Figure 2: Block diagram of `isValid`

The implementation:

```java
// returns true if every "(" has a matching ")" in the correct order
public boolean isValid(String input) {
    Stack<Integer> openIndexes = new Stack<>(); // index of every "(" not matched yet
    boolean valid = true;

    for (int i = 0; i < input.length(); i++) {
        char ch = input.charAt(i);

        if (ch == '(') {
            openIndexes.push(i);
            logger.trace("Opening parenthesis found at index " + i);
        } else if (ch == ')') {
            if (openIndexes.isEmpty()) {
                // there is no "(" before this one
                valid = false;
                logger.warn("Closing parenthesis at index " + i + " has no opening parenthesis");
            } else {
                int start = openIndexes.pop();
                logger.info("Matching pair found: indexes " + start + " and " + i);
            }
        }
    }
```

```
23
24          // a "(" that is still on the stack was never closed
25          if (!openIndexes.isEmpty()) {
26              valid = false;
27          }
28
29          logger.info("Result for \"" + input + "\": " + valid);
30          if (!valid) {
31              logger.error("\"" + input + "\" is invalid");
32          }
33          return valid;
34      }
```

8 Testing and Correctness

8.1 Valid and invalid input

A valid input is `(())((()())())` and the result is `true`. An invalid input is `)(((()))` and the result is `false`.

8.2 How the program decides

The input is valid only if two conditions are true at the same time: no closing parenthesis appears when the stack is empty, and the stack is empty at the end of the string.

8.3 A test that prevented an error

The input `)(((()))` was the most useful one. It contains the same number of `(` and `)` characters, so a solution that only counts the parentheses would accept it. The input is invalid because the first `)` comes before any `(`. This test is the reason why the program checks for an empty stack when a `)` appears. The inputs `(()` and `())` test the two kinds of error separately, and the empty string is valid because there is nothing to match.

Input	Expected	Obtained	Status
`()`	true	true	passed
`)(((()))`	false	false	passed
`(`	false	false	passed
`(())((()())())`	true	true	passed
`(()`	false	false	passed
`())`	false	false	passed
`""` (empty)	true	true	passed

8.4 Effect of the log level

All the tests were run twice, with the level TRACE and with the level INFO. The results were the same (7 of 7 passed), which shows that the level changes only what is printed, not the validation. The output for the input `)(()))` with the level INFO is:

```
[10:11:39] [WARN] Parentheses: Closing parenthesis at index 0 has no opening parenthesis
[10:11:39] [INFO] Parentheses: Matching pair found: indexes 3 and 4
[10:11:39] [INFO] Parentheses: Matching pair found: indexes 2 and 5
[10:11:39] [INFO] Parentheses: Matching pair found: indexes 1 and 6
[10:11:39] [INFO] Parentheses: Result for ")((()))": false
[10:11:39] [ERROR] Parentheses: ")((()))" is invalid
```

With the level TRACE the same run also prints the three TRACE lines for the opening parentheses found at the indexes 1, 2 and 3. With the level ERROR and the format `%l - %n - %m` only one line appears:

```
ERROR - Parentheses - "(()" is invalid
```

9 Extra Credit — the %r Placeholder

The placeholder `%r` is replaced with a random message that belongs to the level of the current entry. The messages are kept in a static two-dimensional array, with one row for every level (the row number is the `ordinal()` of the level), and one message from the row is chosen with `Random`.

```
1  private static final String[][] RANDOM_MESSAGES = {
2     {"Stepping through the input", "Still tracing", "Checking next character
          "},
3     {"Operation completed", "Everything looks good", "Here is your result"},
4     {"Something looks odd", "Proceed with caution", "Unexpected, but not
          fatal"},
5     {"Operation failed", "Something went wrong", "Invalid input detected"}
6  };
7
8  // inside getValue:
9  String[] options = RANDOM_MESSAGES[msgLevel.ordinal()];
10 return options[random.nextInt(options.length)];
```

With the format `[%t] [%l] %r` one possible output is:

```
[10:11:39] [TRACE] Stepping through the input
[10:11:39] [INFO] Everything looks good
[10:11:39] [WARN] Proceed with caution
[10:11:39] [ERROR] Something went wrong
```

10 Reflection

The most important concept I applied was the difference between static and instance members, together with the separation of responsibilities. The level and the format are shared by the class,

the name belongs to every object, and the validator uses the logger without knowing how the printing works.

With more time I would improve the following:

- the logger would write to a file or another output, not only to the console. For this the printing should be moved into a separate method or class;
- I would write the tests with JUnit instead of my own `runTests` method, which compares the results manually;
- the validator would receive the logger in its constructor, instead of creating it inside. In this way it is easier to test it and to replace the logger.

11 Conclusion

In this laboratory work I implemented a `Logger` class with a configurable level and format, and a `ParenthesesValidator` class that uses it. The logger filters the messages by level and replaces the placeholders in a single pass, and the validator checks the parentheses with a stack in linear time. All tests from the handout pass, with any configured level, and the extra placeholder `%r` works for every log level.

A Complete Source Code

A.1 LogLevel.java

```
// the order of the constants is the severity (TRACE is the lowest, ERROR
    the highest)
public enum LogLevel {
    TRACE,
    INFO,
    WARN,
    ERROR
}
```

A.2 Logger.java

```
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.Random;

public class Logger {
    // static: shared by all loggers (they belong to the class, not to an
        object)
    private static LogLevel level = LogLevel.INFO;
    private static String format = "[%t] [%l] %n: %m";

```

```
10      private static final DateTimeFormatter TIME_FORMAT = DateTimeFormatter.
            ofPattern("HH:mm:ss");
11      private static final Random random = new Random();
12
13      // random messages for %r: one row per level, in the same order as
            LogLevel (TRACE, INFO, WARN, ERROR)
14      private static final String[][] RANDOM_MESSAGES = {
15          {"Stepping through the input", "Still tracing", "Checking next
                character"},
16          {"Operation completed", "Everything looks good", "Here is your result
                "},
17          {"Something looks odd", "Proceed with caution", "Unexpected, but not
                fatal"},
18          {"Operation failed", "Something went wrong", "Invalid input detected"
                }
19      };
20
21      // instance: every logger has its own name
22      private final String name;
23
24      public Logger(String name) {
25          this.name = name;
26      }
27
28      public static void setLevel(LogLevel newLevel) {
29          level = newLevel;
30      }
31
32      public static void setFormat(String newFormat) {
33          format = newFormat;
34      }
35
36      public void log(LogLevel msgLevel, String message) {
37          // ignore messages that are less severe than the configured level
38          if (msgLevel.ordinal() < level.ordinal()) {
39              return;
40          }
41          System.out.println(buildText(msgLevel, message));
42      }
43
44      public void trace(String message) {
45          log(LogLevel.TRACE, message);
46      }
47
48      public void info(String message) {
49          log(LogLevel.INFO, message);
50      }
51
52      public void warn(String message) {
53          log(LogLevel.WARN, message);
54      }
55
56      public void error(String message) {
57          log(LogLevel.ERROR, message);
58      }
59
60      // goes through the format once and replaces every placeholder
```

```
61      // (one pass, so a "%n" written inside the message is not replaced again
            )
62      private String buildText(LogLevel msgLevel, String message) {
63          String result = "";
64          int i = 0;
65          while (i < format.length()) {
66              char c = format.charAt(i);
67              String value = null;
68              if (c == '%' && i + 1 < format.length()) {
69                  value = getValue(format.charAt(i + 1), msgLevel, message);
70              }
71              if (value != null) {
72                  result += value;
73                  i += 2; // skip the % and the letter
74              } else {
75                  result += c; // normal character, print it unchanged
76                  i++;
77              }
78          }
79          return result;
80      }
81
82      // returns the text for a placeholder letter, or null if the letter is
            not a placeholder
83      private String getValue(char letter, LogLevel msgLevel, String message)
            {
84          if (letter == 't') {
85              return LocalTime.now().format(TIME_FORMAT);
86          } else if (letter == 'm') {
87              return message;
88          } else if (letter == 'n') {
89              return name;
90          } else if (letter == 'l') {
91              return msgLevel.name();
92          } else if (letter == 'r') {
93              String[] options = RANDOM_MESSAGES[msgLevel.ordinal()];
94              return options[random.nextInt(options.length)];
95          }
96          return null;
97      }
98  }
```

A.3 ParenthesesValidator.java

```
1   import java.util.Stack;
2
3   public class ParenthesesValidator {
4       private final Logger logger = new Logger("Parentheses");
5
6       // returns true if every "(" has a matching ")" in the correct order
7       public boolean isValid(String input) {
8           Stack<Integer> openIndexes = new Stack<>(); // index of every "(" not
                matched yet
9           boolean valid = true;
10
```

```
        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);

            if (ch == '(') {
                openIndexes.push(i);
                logger.trace("Opening parenthesis found at index " + i);
            } else if (ch == ')') {
                if (openIndexes.isEmpty()) {
                    // there is no "(" before this one
                    valid = false;
                    logger.warn("Closing parenthesis at index " + i + " has no
                        opening parenthesis");
                } else {
                    int start = openIndexes.pop();
                    logger.info("Matching pair found: indexes " + start + " and
                        " + i);
                }
            }
        }

        // a "(" that is still on the stack was never closed
        if (!openIndexes.isEmpty()) {
            valid = false;
        }

        logger.info("Result for \"" + input + "\": " + valid);
        if (!valid) {
            logger.error("\"" + input + "\" is invalid");
        }
        return valid;
    }
}
```

A.4 Main.java

```
public class Main {

    private static void runTests() {
        ParenthesesValidator validator = new ParenthesesValidator();
        String[] inputs = {"()", ")((()))", "(", "(())((()())())", "(()", "()
            )", ""};
        boolean[] expected = {true, false, false, true, false, false, true};

        int passed = 0;
        for (int i = 0; i < inputs.length; i++) {
            boolean result = validator.isValid(inputs[i]);
            String status;
            if (result == expected[i]) {
                passed++;
                status = "PASS";
            } else {
                status = "FAIL";
            }
            System.out.println(status + " input=\"" + inputs[i]
                  + "\" expected=" + expected[i] + " got=" + result + "\n");
```

```
        }
        System.out.println(passed + "/" + inputs.length + " tests passed\n");
    }

    public static void main(String[] args) {
        System.out.println("=== Level TRACE ===");
        Logger.setLevel(LogLevel.TRACE);
        runTests();

        System.out.println("=== Level INFO (no trace messages) ===");
        Logger.setLevel(LogLevel.INFO);
        runTests();

        System.out.println("=== Level ERROR, custom format ===");
        Logger.setLevel(LogLevel.ERROR);
        Logger.setFormat("%l - %n - %m");
        new ParenthesesValidator().isValid("(()");

        System.out.println("\n=== Extra credit: %r ===");
        Logger.setLevel(LogLevel.TRACE);
        Logger.setFormat("[%t] [%l] %r");
        Logger demo = new Logger("Demo");
        demo.trace("ignored text");
        demo.info("ignored text");
        demo.warn("ignored text");
        demo.error("ignored text");
    }
}
```